

Use SMOTE for address class imbalance

```
!pip install imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in /usr/local/lib/python3.12/dis
Requirement already satisfied: numpy<3,>=1.25.2 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy<2,>=1.11.4 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scikit-learn<2,>=1.4.2 in /usr/local/lib/python3.
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in /usr/local/lib/pyth
Requirement already satisfied: joblib<2,>=1.2.0 in /usr/local/lib/python3.12/dis
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in /usr/local/lib/python3
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_s

from imblearn.over_sampling import SMOTE
```

Start coding or [generate](#) with AI.

```
df = pd.read_csv("Customer Churn new.csv")
df.head()
```

	RowNumber	CustomerId	Surname	CreditScore	Geography	Gender	Age	Tenure
0	747	15787619	Hsieh	844	France	Male	18	2
1	1620	15770309	McDonald	656	France	Male	18	10
2	1679	15569178	Kharlamov	570	France	Female	18	4
3	2022	15795519	Vasiliev	716	Germany	Female	18	3
4	2137	15621893	Bellucci	727	France	Male	18	4

Next steps: [Generate code with df](#) [New interactive sheet](#)

```
#Preprocessing
# Drop unnecessary columns
```

```
df.drop(['RowNumber', 'CustomerId', 'Surname'], axis=1, inplace=True)

# Handle missing values
df = df.dropna()

# Encode categorical variables
le = LabelEncoder()

for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

```
#Features & Target
X = df.drop('Exited', axis=1)
y = df['Exited']
```

```
#Stratified Split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42
)

X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.6, stratify=y_temp, random_state=42
)
```

```
#Apply SMOTE (ONLY on training data
smote = SMOTE(random_state=42)

X_train_sm, y_train_sm = smote.fit_resample(X_train, y_train)

print("Before SMOTE:", y_train.value_counts())
print("After SMOTE:", pd.Series(y_train_sm).value_counts())
```

```
Before SMOTE: Exited
0    5972
1    1528
Name: count, dtype: int64
After SMOTE: Exited
1    5972
0    5972
Name: count, dtype: int64
```

```
rf_model = RandomForestClassifier(
    n_estimators=200,
    max_depth=10,
    random_state=42
)

rf_model.fit(X_train_sm, y_train_sm)
```

▼ RandomForestClassifier ⓘ ?

```
RandomForestClassifier(max_depth=10, n_estimators=200, random_state=42)
```

```
y_val_pred = rf_model.predict(X_val)

print("Validation Results:")
print("Accuracy:", accuracy_score(y_val, y_val_pred))
print("Precision:", precision_score(y_val, y_val_pred))
print("Recall:", recall_score(y_val, y_val_pred))
print("F1 Score:", f1_score(y_val, y_val_pred))
```

Validation Results:
Accuracy: 0.742
Precision: 0.4075342465753425
Recall: 0.5833333333333334
F1 Score: 0.4798387096774194

```
y_test_pred = rf_model.predict(X_test)

print("\nTest Results:")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred))
print("Recall:", recall_score(y_test, y_test_pred))
print("F1 Score:", f1_score(y_test, y_test_pred))
```

Test Results:
Accuracy: 0.726
Precision: 0.38009049773755654
Recall: 0.5508196721311476
F1 Score: 0.4497991967871486

```
y_test_pred = rf_model.predict(X_test)

print("\nTest Results:")
print("Accuracy:", accuracy_score(y_test, y_test_pred))
print("Precision:", precision_score(y_test, y_test_pred))
print("Recall:", recall_score(y_test, y_test_pred))
print("F1 Score:", f1_score(y_test, y_test_pred))
```

Test Results:
Accuracy: 0.726
Precision: 0.38009049773755654
Recall: 0.5508196721311476
F1 Score: 0.4497991967871486

```
cm = confusion_matrix(y_test, y_test_pred)
print("\nConfusion Matrix:\n", cm)
```

```
Confusion Matrix:
[[921 274]
 [137 168]]
```

```
print("\nClassification Report:\n")
print(classification_report(y_test, y_test_pred))
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.87	0.77	0.82	1195
1	0.38	0.55	0.45	305
accuracy			0.73	1500
macro avg	0.63	0.66	0.63	1500
weighted avg	0.77	0.73	0.74	1500

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

